

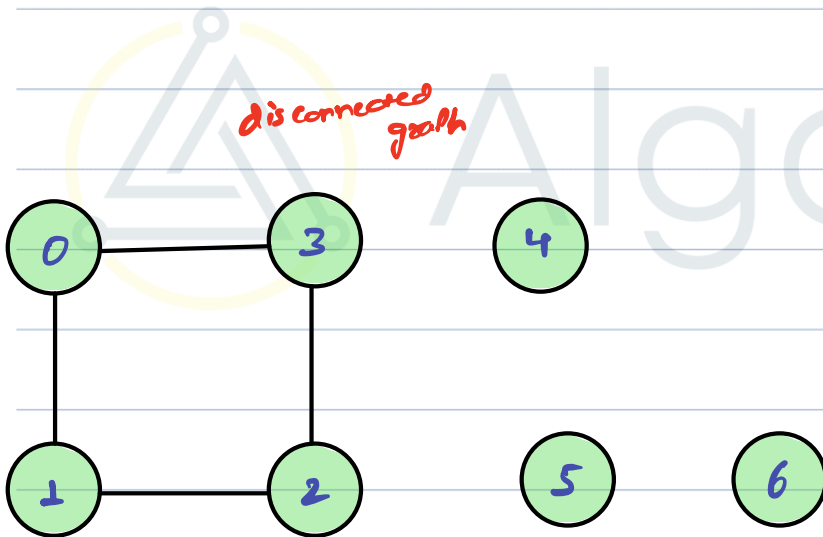


Today's agenda

↳ BFS Problem

↳ Rotting oranges

↳ o/s

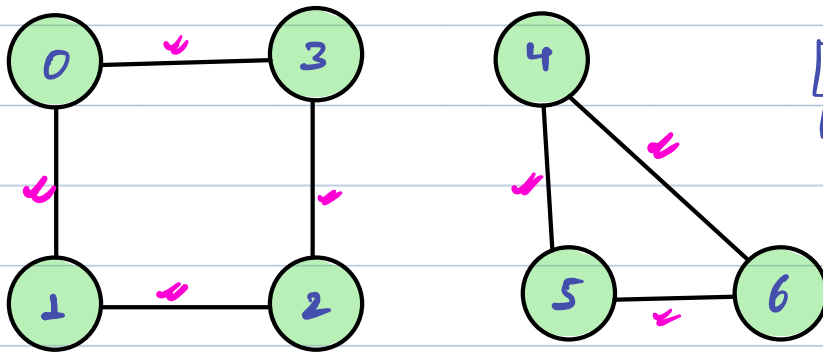


$n=7$ $m=4$

0	1
1	2
2	3
0	3



Q) Given undirected graph, Source node and destination node. Check if destination node can be visited from Source node or not?



graph

0	1	2	3	4	5	6
1 3	2 0	1 3	2 0	6 5	4 6	4 5

SN: 0 DN: 2 → true

SN: 0 DN: 4 → False

SN: 0 DN: 4

graph

0	1	2	3	4	5	6
1 3	2 0	1 3	2 0	6 5	4 6	4 5

Q:

0	1	2	3	4	5	6
1	2	1	2	3	3	3

n nodes

E:

N: 7 M: 7

0	1
1	2
2	3
0	3
4	5
5	6
4	6



Pseudo Code

no of nodes
no of edges
boolean BFS (int n , int m , int edges[m][2], int sn , int dn) {

List<List<Integer>> graph = construction($n, m, edges$);

Queue<Integer> q;

boolean[] vis = new boolean [n];

q.add(sn);

vis[sn] = true;

while (q.size() > 0) {

int xem = q.remove();

S.o.p(xem);

// add unvisited nodes

List<Integer> nbs = graph.get(xem);

for (int v: nbs) {

if (vis[v] == false) {

q.add(v);

vis[v] = true;

}

}

}

return vis[dn];

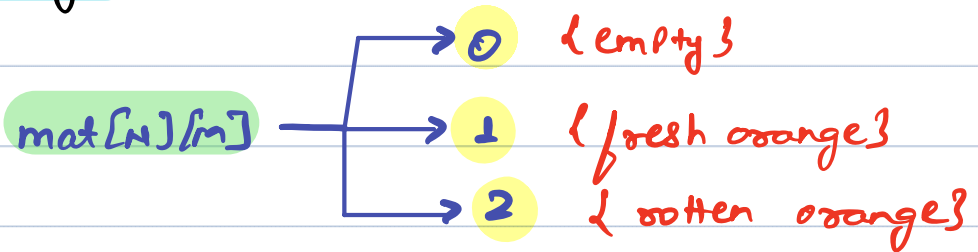
}

T.C: $O(n+m)$ or $O(V+E)$

S.C: $O(V+E)$

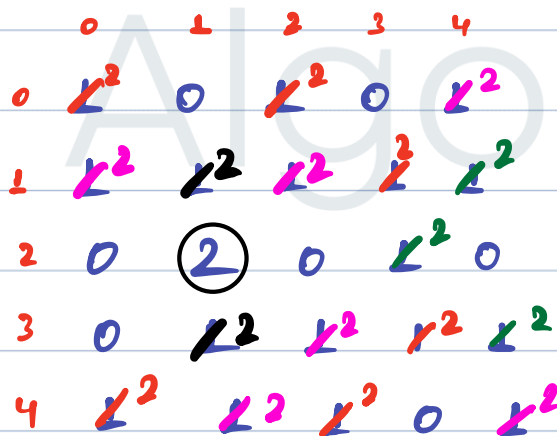


Q) Rotting oranges



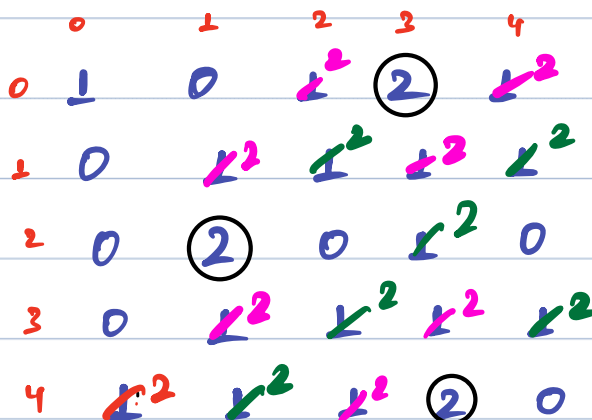
↳ Every minute any fresh orange adjacent to rotten orange becomes rotten. Find min time when all oranges become rotten. if not possible to rot all oranges, return -1

Ex 1:



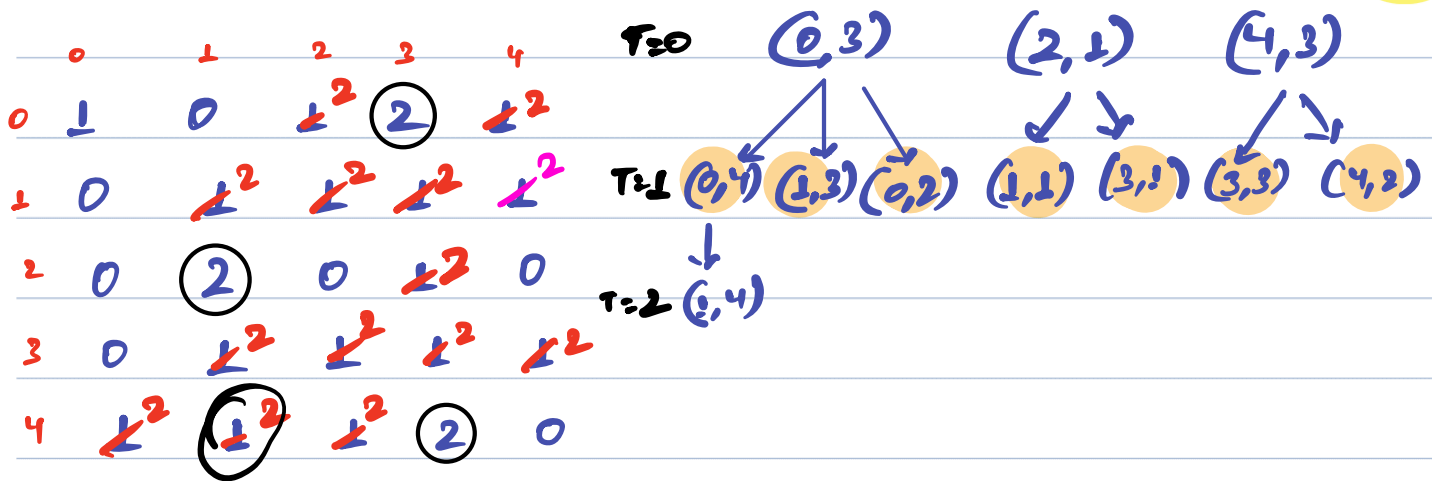
T=0
↓
T=1
↓
T=2
↓
T=3
↓
T=4
↓
T=5

Ex 2:

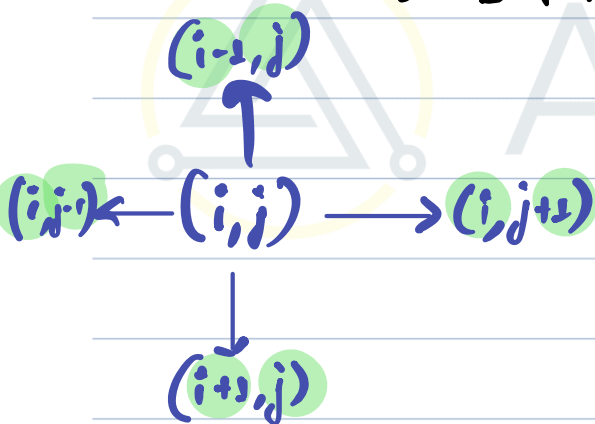


T=0
↓
T=1
↓
T=2
↓
T=3

ans = -1



<Pair> q: ~~(0,2,0)~~ ~~(2,1,0)~~ ~~(4,2,0)~~ ~~(0,1,1)~~ ~~(1,3,1)~~ ~~(0,2,1)~~ ~~(1,1,1)~~ ~~(2,1,1)~~
~~(3,3,1)~~ ~~(4,2,1)~~ ~~(1,1,2)~~ ~~(2,0,2)~~ ~~(1,0,2)~~ ~~(2,0,2)~~ ~~(4,1,2)~~
~~(3,4,2)~~ ~~(1,0,3)~~
 rem = (4,0,3)



↓ Action - 1

Class Pairs

int i
ind j
int t



11 Pseudo code

```
int rottenOranges (int mat[N][M]) {  
    Queue <Pair> q;  
  
    for (int i=0; i<N; i++) {  
        for (int j=0; j<M; j++) {  
            if (mat[i][j] == 2) {  
                q.add (new Pair (i, j, 0));  
            }  
        }  
    }  
  
    int ans = -1;  
    while (q.size() > 0) {  
        Pair rem = q.remove();  
        int row = rem.i;  
        int col = rem.j;  
        int ctime = rem.t;  
        ans = ctime;  
        // row-1, col  
        if (row-1 >= 0 && mat[row-1][col] == 1) {  
            q.add (new Pair (row-1, col, ctime+1));  
            mat[row-1][col] = 2;  
        }  
    }  
}
```

T.C: $O(N \times M)$

S.C: $O(N \times M)$



```
// row, col+1
```

```
if (col+1 < m && mat[row][col+1] != 1) {  
    q.add(new Pair(row, col+1, time+1));  
    mat[row][col+1] = 1;  
}
```

```
// row+1, col
```

```
// row, col-1
```

```
}
```

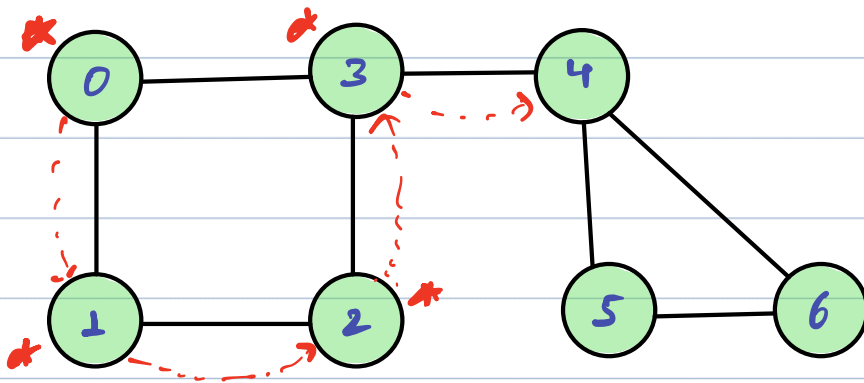
```
for (int i=0; i<n; i++) {  
    for (int j=0; j<m; j++) {  
        if (mat[i][j] == 1) {  
            return -1;  
        }  
    }  
}
```

```
return ans;
```

```
}
```

break till 10:40 pm

Dfs → Depth first search Traversal



→ traversal in any order

graph

0	1	2	3	4	5	6
1 2	0 2	1 3	0 2 4	6 5 3	4 6	4 5

src

0

nbx

1 3

v=1

1

0 2

v=2

2

1 3

v=3

3

0 2 4

v=4



Java code

```
void dfs (List<List<Integer>> graph, boolean vis[], int src) {  
  
    List<Integer> nbos = graph.get(src);  
  
    for (int v: nbos) {  
        if (vis[v] == false) {  
            vis[v] = true;  
            dfs (graph, vis, v);  
        }  
    }  
}
```

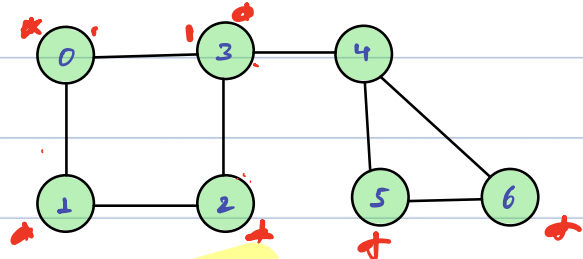


```
void dfs (List<List<Integer>> graph, boolean vis[], int src)
```

~~if (all nodes are visited) { return; }~~

```
List<Integer> nbws = graph.get(src);
```

```
for (int v: nbws) {  
    if (vis[v] == false) {  
        vis[v] = true;  
        dfs(graph, vis, v);  
    }  
}
```



graph

0	1	2	3	4	5	6
3 1	0 2	1 3	0 2 4	6 5 3	4 6	4 5

0	1	2	3	4	5	6
True	True	True	True	True	True	True

Src	nbws	Count	it's 1 → v: 3 2 → v: 2
0	3 1	2	
3	0 2 4	3	1 → v: 0 2 → v: 2 3 → v: 4
2	1 3	2	1 → v: 1 2 → v: 3
1	0 2	2	1 → v: 0 2 → v: 2
4	5 6 3	3	1 → v: 5 2 → v: 6 3 → v: 3
5	4 6	2	1 → v: 4 2 → v: 6
6	4 5	2	1 → v: 4 2 → v: 5